How-To Library

Informatica

Replacing User Exits in IDD Applications

© Copyright Informatica LLC 2017, 2023. Informatica and the Informatica logo are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html.

Abstract

To use the new REST APIs and data views in Informatica Data Director (IDD) 10.1 and later, you must migrate your applications from the subject area data model to the business entity data model. You cannot migrate user exits, but you can replicate the behavior of some types of user exits by using external calls. This document tells you which user exits you can replace with external calls, and walks through how to create a sample external call.

Before you begin, we highly recommend that you read *Migrating IDD Applications to the Business Entity Data Model*.

Supported Versions

- MDM Multidomain Edition 10.1 and later

Table of Contents

Introduction. . . . 2
Types of User Exits that You Can Replace. . . . 3
Key Concepts. . . . 3
Business Entity. . . . 3
Business Entity View. . . . 3
Business Entity Services. . . . 3
External Calls. . . . 4
Transformations. . . . 4
How to Create External Calls. . . . 4
Example Code. . . . 5
Understand and Deploy the Example Web Service. . . . 5
Log into the Provisioning Tool. . . . 6
Upload the WSDL file. . . . 6
Register the SOAP Service. . . . 7
Configure the External Call. . . . 7
Publish the Configurations. . . . 8
Test the Address Validation. . . . 8
Test the Merge Logic. . . . 8
Other Ways to Customize the Behavior of an IDD Application. . . . 13

Introduction

A user exit is a way to customize the behavior of an Informatica Data Director (IDD) application that is based on the subject area data model. In IDD version 10.1, IDD was modernized to work with REST APIs (also known as business entity services) and new versions of the IDD views. To enable this modernization, the subject area data model was replaced with the business entity data model. User exits do not work with the business entity data model.

To achieve results that are similar to user exits, you can extend the business entity services by using external calls. An external call is a request to a web service. In the web service, you define the operations or custom logic that you want to run. When you define an external call, you specify the business entity service and the service phase at which you want to trigger the external call.

We'll start by identifying the types of user exits that you can replace. Then we'll review some key concepts and walk through an example. After the walkthrough, we will briefly discuss other ways that you can customize an IDD application, UI components, and data as a service providers.

Types of User Exits that You Can Replace

You can replace user exits that perform the following tasks:

- Cleanse data by using an external service
- Validate data by using an external service, and return custom messages
- Run custom logic when a service reaches a specified event

Note: The business entity services and service phases are not the same as the operations and entry points that are used to define user exits. Review the list of "External Calls" on page 4 to determine if they meet your needs.

Key Concepts

The business entity data model introduced some new concepts. Before you begin, review the following concepts.

Business Entity

A business entity is a model of a person, place, or thing that is important to your organization. For example, you might define customers, locations, or products. A business entity model defines all the fields that you want to include in your master data, such as name fields or address fields. Business entity models replace subject area configurations.

You create the business entity models when you migrate an IDD application to the BE data model. For more information, see *Migrating IDD Applications to the Business Entity Data Model*.

Business Entity View

A business entity view is a logical separation between the business entity model and what you want to make available to end users. You select the fields that you want to make available in the view. Define business entity views when you have external systems or stakeholders who want to see the data in a different structure from the business entity model. You can also define business entity views to hide sensitive data fields from some users and show them to other users.

For more information about business entity views, see the *Informatica MDM Multidomain Edition Provisioning Tool Guide*.

Business Entity Services

A business entity service performs an operation, such as reading data, writing data, and searching data. A business entity service has a set of service steps. An incoming request passes through each service step.

Note: Business entity services replace the Services Integration Framework (SIF) that worked with the subject area data model.

For more information about business entity services, see the *Multidomain MDM Business Entity Services Guide*.

External Calls

External calls are extensions to the business entity services. When you define an external call, you extend a business entity service and trigger the call at a service phase. The external call sends a request to the specified web service to cleanse, validate, or transform data.

Based on the type of business entity service, you can run an external call before the service starts, before the validation starts, after the validation ends, or as the service ends.

The following list identifies the business entity services that you can extend and the service phases:

- Read services
  - ReadCO.BeforeEverything
  - ReadCO.AfterEverything
  - ReadView.BeforeEverything
  - ReadView.AfterEverything
- Write services
  - WriteCO.BeforeEverything
  - WriteCO.BeforeValidate
  - WriteCO.AfterValidate
  - WriteCO.AfterEverything
  - WriteView.BeforeEverything
  - WriteView.BeforeValidate
  - WriteView.AfterValidate
  - WriteView.AfterEverything
- Merge services
  - MergeCO.BeforeEverything
  - MergeCO.AfterEverything
  - PreviewMergeCO.BeforeEverything
  - PreviewMergeCO.AfterEverything

For more information about external calls, see the *Informatica MDM Multidomain Edition Provisioning Tool Guide*.

Transformations

Use transformations to implement cleanse functions on fields for cleansing and validation. You can choose to map the fields directly with no change to the data, or you can create cleanse functions that run on the data in a field. The transformations run during read and write operations.

For more information about transformations, see the *Informatica MDM Multidomain Edition Provisioning Tool Guide*.

How to Create External Calls

Let's walk through how to create an external call. This tutorial uses the BESExternalCall example from the MDM Resource Kit. We will deploy the example web service and then use the Provisioning tool to configure the web service and create the external call for this example.

In this tutorial, we perform the following tasks:

1. Understand and deploy the example Web service

2. Log into the Provisioning tool
3. Upload the WSDL file
4. Register the SOAP service
5. Configure external calls
6. Publish the changes to the MDM Hub
7. Test the external calls

Example Code

This tutorial uses the BESExternalCall example from the MDM Hub Resource Kit.

The external call to the web service can perform the following actions:

- When creating or updating a person, validate that the person has at least one address. If the validation fails, return a customized error message.
- When generating a preview of merged records, do not merge telephone numbers
- When merging records, do not merge telephone numbers.

The following table describes the files that support the BESExternalCall example and tells you where to find them:

File Name	Description	Location
`bes-external-call.xsd`	Defines the request and response types for all external calls. An external web service operation must use the request and response types for the input and output elements.	`<MDM installation directory>/hub/server/lib/mdm-spi.jar`
`custom-logic-service.wsdl`	WSDL file for the example web service. Defines the external services, operations, methods, and data types that the service methods exchange.	`<MDM installation directory>/hub/resourcekit/samples/BESExternalCall/source/resources/webapp/WEB-INF/wsdl/`
`ValidatePerson.java` and `MergePerson.java`	Examples of custom validate and merge operations that run with the example web service. **Note:** This example uses .java files, but your external web service might require a different coding standard.	`<MDM installation directory>/hub/resourcekit/samples/BESExternalCall/source/java/`
`build.xml`	Builds the `bes-external-call.ear` file.	`<MDM installation directory>/hub/resourcekit/samples/BESExternalCall/`
`bes-external-call.ear`	The .ear file that you deploy to the application server to run this example.	*Generated file*

Understand and Deploy the Example Web Service

The example contains a WSDL file and the custom logic (Java files) used by the external calls. Review the files in the example. To use the example, build and deploy the .ear file to the application server that runs the MDM Hub.

Tip: When designing your own external calls, first develop the custom logic in the framework of the external service. Then create the WSDL file that defines the external services, operations, methods, and data types that the service

methods exchange. Ensure that the WSDL file uses the request and response types that are defined in the `bes-external-call.xsd` file.

1. To understand the requirements for the request and response types, review the `bes-external-call.xsd` file.
2. To understand how we implemented the request and response types for the example web service, review the `custom-logic-service.wsdl` file.
3. Optionally, to understand the custom logic used with the example web service, review the Java files in the Resource Kit.
4. To build the example .ear file, use the `build.xml` file with an ANT build tool.
5. Deploy the `bes-external-call.ear` file to the application server that runs the MDM Hub.

Log into the Provisioning Tool

We configure the web service and external call in the Provisioning tool. To log in, you need the URL for the Provisioning tool and your user credentials. The URL contains the MDM Hub server host name and port number. If you do not have this information, contact your MDM Hub administrator. For this example, when you are prompted to select an Operational Reference Store (ORS), select the Sample MDM ORS.

The MDM Hub must be running on the same application server as the Provisioning tool.

1. Open a supported browser.
2. Enter the URL for the Provisioning tool. The URL has the following format:

   Secure connections. `https://<MDM Hub Server host name>:<MDM Hub Server port number>/provisioning/`

   Non-secure connections. `http://<MDM Hub Server host name>:<MDM Hub Server port number>/provisioning/`

   The **Log In** page opens.
3. Enter your user name and password.
4. Click **Log In**.
5. When prompted for an ORS, select MDM Sample.

   The Provisioning tool opens and displays the **Home** page.

Upload the WSDL file

Upload the WSDL file for the BESExternalCall example web service.

1. In the Provisioning tool, click **Business Entity** > **Extensions**.

   The **Extensions** page opens.
2. From the **Extensions** list, select **WSDL**, and then click **Create**.
3. In the properties pane, specify the following properties:

Property	Description	Example Values
Name	A descriptive name for the example web service.	beexternal
URL	URL of the WSDL file.	`http://<host>:<port>/bes-external-call/customLogicService?wsdl`

4. Click **Apply**.

   The tree view displays the details.

Register the SOAP Service

Register the SOAP service and operations that the WSDL describes.

1. In the **Extensions** page, from the **Extensions** list, select **SOAP Services**, and then click **Create**.

   The properties appear in the properties panel.
2. In the properties panel, specify the following properties:

Property	Description	Example Values
Name	Name for the SOAP service.	userexit
WSDL	Name that you specified when you uploaded the example WSDL file. The details of the web service appear in the Namespace, Service Name, Port Name, and Endpoint Address fields.	besexternal
SOAP Header	Any additional information that you require to run the web service, such as user name and password.	*n/a*

3. Click **Apply**.

   The tree view displays the details.

Configure the External Call

Next, we configure the external call. The external call uses the configured SOAP service and operations, and the call is triggered for three service phases.

1. In the **Extensions** page, from the **Extensions** list, select **External Calls**, and then click **Create**.
2. In the properties panel, set the following properties:

Properties	Description	Example Values
Name	Name for the external call.	validateperson
SOAP service	SOAP service that you configured.	userexit
SOAP operation	SOAP operation to perform.	validate

3. Click **Apply**.

   The Tree View is populated based on the content of the SOAP service and operation.
4. In the Tree View, click **businessEntity**, click **Create**, and select the **Person** business entity.
5. Click **servicePhase**, click **Create**, and select the following service phases:
   - WriteCO.BeforeValidate
   - PreviewMergeCO.BeforeEverything

- MergeCO.BeforeEverything

6. Click **Apply**.

Publish the Configurations

The configurations for the WSDL file, SOAP service, and external calls are stored in a cache. Publish the configurations to the MDM Hub.

▶ In the Provisioning tool, click **Publish**.

The publish process saves the configurations to the MDM Hub.

Test the Address Validation

We can use the REST APIs to verify that the validation step works. Create a person without an address. If the external call is set up correctly, a validation error appears that says that a person must have at least one address.

To use the REST URLs, we need the following items: the database ID for the MDM Sample ORS and an MDM user name.

1. Find the database ID for the MDM Sample ORS:
   a. Log into the Hub Console.
   b. In the **Configuration** workbench, click **Databases**.
   c. Select the MDM Sample database.
   d. In the Database Properties panel, copy the value for Database ID. Use this value in the REST URL.
2. Use the Create API to create a Person business entity without an address.

```
POST http://<host>:<port>/cmx/cs/<database id>/Person?systemName=<user name>
{
        firstName: "John"
    }
```

   The validation test fails and displays the error. The person is not created.
3. Create a Person business entity with an address.

```
POST http://<host>:<port>/cmx/cs/<database id>/Person?systemName=<user name>
{
        firstName: "John",
        Addresses: {
            item: [
                {
                    cityName: "Toronto"
                }
            ]
        }
    }
```

   The validation test passes without errors. The person is created.

Test the Merge Logic

Use the MergePreview API to test the merge logic that prevents telephone numbers from merging.

1. Use the Create API to create two Person business entities.

```
POST http://<host>:<port>/cmx/cs/<ors>/Person?systemName=Admin
    {
        firstName: "John",
        Addresses: {
            item: [
                {
```

```
                    cityName: "Toronto"
                }
            ]
        },
        TelephoneNumbers: {
            item:[
                {
                    phoneNum: "111-11-11"
                }
            ]
        }
    }
POST http://<host>:<port>/cmx/cs/<ors>/Person?systemName=Admin
    {
        firstName: "John",
        Addresses: {
            item: [
                {
                    cityName: "Ottawa"
                }
            ]
        },
        TelephoneNumbers: {
            item:[
                {
                    phoneNum: "222-22-22"
                }
            ]
        }
    }
```

The response contains the rowIDs, for example:

- Person: 161923, 161924
- Addresses: 2123, 2124
- TelephoneNumbers: 101723, 101724

2. Run the PreviewMerge API to preview the merged Person business entities.

```
POST http://<host>:<port>/cmx/cs/<ors>/Person/161923?action=previewMerge&depth=2
{
        keys: [
            {
                rowid: "161924"
            }
        ]
    }
```

The response contains two addresses and two telephone numbers.

```
{
  "Person": {
    "rowidObject": "161923        ",
    "creator": "admin",
    "createDate": "2016-10-20T09:50:35.878-04:00",
    "updatedBy": "admin",
    "lastUpdateDate": "2016-10-20T09:50:35.879-04:00",
    "consolidationInd": 4,
    "lastRowidSystem": "SYS0          ",
    "hubStateInd": 1,
    "label": "Person: , Bill",
    "partyType": "Person",
    "displayName": "Bill",
    "firstName": "Bill",
   "TelephoneNumbers": {
      "link": [],
      "firstRecord": 1,
      "recordCount": 2,
      "pageSize": 2,
```

```
  "item": [
    {
      "rowidObject": "101723        ",
      "creator": "admin",
      "createDate": "2016-10-20T09:50:35.904-04:00",
      "updatedBy": "admin",
      "lastUpdateDate": "2016-10-20T09:50:35.905-04:00",
      "consolidationInd": 4,
      "lastRowidSystem": "SYS0          ",
      "hubStateInd": 1,
      "label": "PhoneNumbers",
      "phoneNum": "111-1111     ",
      "phoneCountryCd": "1"
    },
    {
      "rowidObject": "101724        ",
      "creator": "admin",
      "createDate": "2016-10-20T09:50:54.768-04:00",
      "updatedBy": "admin",
      "lastUpdateDate": "2016-10-20T09:50:54.769-04:00",
      "consolidationInd": 4,
      "lastRowidSystem": "SYS0          ",
      "hubStateInd": 1,
      "label": "PhoneNumbers",
      "phoneNum": "222-2222     ",
      "phoneCountryCd": "1"
    }
  ]
},
"Addresses": {
  "link": [],
  "firstRecord": 1,
  "recordCount": 2,
  "pageSize": 2,
  "item": [
    {
      "rowidObject": "2123          ",
      "creator": "admin",
      "createDate": "2016-10-20T09:50:37.956-04:00",
      "updatedBy": "admin",
      "lastUpdateDate": "2016-10-20T09:50:37.956-04:00",
      "consolidationInd": 4,
      "lastRowidSystem": "SYS0          ",
      "hubStateInd": 1,
      "label": "Addresses",
      "Address": {
        "rowidObject": "2121          ",
        "creator": "admin",
        "createDate": "2016-10-20T09:50:36.922-04:00",
        "updatedBy": "admin",
        "lastUpdateDate": "2016-10-20T09:50:37.923-04:00",
        "consolidationInd": 4,
        "lastRowidSystem": "SYS0          ",
        "hubStateInd": 1,
        "label": "Address",
        "cityName": "Toronto"
      }
    },
    {
      "rowidObject": "2124          ",
      "creator": "admin",
      "createDate": "2016-10-20T09:50:54.790-04:00",
      "updatedBy": "admin",
      "lastUpdateDate": "2016-10-20T09:50:54.790-04:00",
      "consolidationInd": 4,
      "lastRowidSystem": "SYS0          ",
      "hubStateInd": 1,
      "label": "Addresses",
      "Address": {
```

```
            "rowidObject": "2122          ",
            "creator": "admin",
            "createDate": "2016-10-20T09:50:54.777-04:00",
            "updatedBy": "admin",
            "lastUpdateDate": "2016-10-20T09:50:54.777-04:00",
            "consolidationInd": 4,
            "lastRowidSystem": "SYS0          ",
            "hubStateInd": 1,
            "label": "Address",
            "cityName": "Ottawa"
          }
        }
      ]
    }
  }
}
```

3. Run the PreviewMerge API with the merge overrides. The overrides trigger the external call that prevents the telephone numbers from merging:

```
POST http://<host>:<port>/cmx/cs/<ors>/Person/161923?action=previewMerge&depth=3
{
    keys: [
        { rowid: "161923" }
    ],
    overrides: {
        Person: {
            Addresses: {
                item:[
                    {
                        rowidObject: "2123",
                        MERGE: {
                            item:[{key:{rowid: "2124"}}],
                            $original: {
                                item:[null]
                            }
                        }
                    }
                ]
            },
            TelephoneNumbers: {
                item:[
                    {
                        rowidObject: "101723",
                        MERGE: {
                            item:[{key:{rowid: "101724"}}],
                            $original: {
                                item:[null]
                            }
                        }
                    }
                ]
            }
        }
    }
}
```

The response contains a single address and two telephone numbers:

```
{
  "Person": {
    "rowidObject": "161923        ",
    "creator": "admin",
    "createDate": "2016-10-20T09:50:35.878-04:00",
    "updatedBy": "admin",
    "lastUpdateDate": "2016-10-20T09:50:35.879-04:00",
    "consolidationInd": 4,
    "lastRowidSystem": "SYS0          ",
    "hubStateInd": 1,
    "label": "Person: , Bill",
    "partyType": "Person",
```

```
    "displayName": "Bill",
    "firstName": "Bill",
   "TelephoneNumbers": {
      "link": [],
      "firstRecord": 1,
      "recordCount": 2,
      "pageSize": 2,
      "item": [
        {
          "rowidObject": "101723        ",
          "creator": "admin",
          "createDate": "2016-10-20T09:50:35.904-04:00",
          "updatedBy": "admin",
          "lastUpdateDate": "2016-10-20T09:50:35.905-04:00",
          "consolidationInd": 4,
          "lastRowidSystem": "SYS0          ",
          "hubStateInd": 1,
          "label": "PhoneNumbers",
          "phoneNum": "111-1111     ",
          "phoneCountryCd": "1"
        },
        {
          "rowidObject": "101724        ",
          "creator": "admin",
          "createDate": "2016-10-20T09:50:54.768-04:00",
          "updatedBy": "admin",
          "lastUpdateDate": "2016-10-20T09:50:54.769-04:00",
          "consolidationInd": 4,
          "lastRowidSystem": "SYS0          ",
          "hubStateInd": 1,
          "label": "PhoneNumbers",
          "phoneNum": "222-2222     ",
          "phoneCountryCd": "1"
        }
      ]
    },
     "Addresses": {
      "link": [],
      "firstRecord": 1,
      "recordCount": 1,
      "pageSize": 1,
      "item": [
        {
          "rowidObject": "2123          ",
          "creator": "admin",
          "createDate": "2016-10-20T09:50:37.956-04:00",
          "updatedBy": "admin",
          "lastUpdateDate": "2016-10-20T09:50:37.956-04:00",
          "consolidationInd": 4,
          "lastRowidSystem": "SYS0          ",
          "hubStateInd": 1,
          "label": "Addresses"
        }
      ]
    },
    "PersonDetails": {
      "link": [],
      "recordCount": 0,
      "pageSize": 0,
      "item": []
    }
  }
}
```

Other Ways to Customize the Behavior of an IDD Application

External calls are only one way to customize an IDD application.

Consider using some of the following features as part of your overall strategy to customize an IDD application:

Action	Feature	Comments
Cleanse or validate data by using Informatica cleanse libraries	Transformations	When you want to cleanse or validate data before a read or write operation, use transformations. You can map fields and specify cleanse functions for the fields. **Tip:** When you need to cleanse data during a different service or service step, create external calls to Informatica Data Quality instead.
Perform an action from a custom button in the user interface	UI External resource components	You can add a component to the Start page or to an entity view. The component can call an external service and display the results. For example, the component can query a CRM system for information about a customer's last contact with the company or the next best offer for the customer. **Note:** Avoid adding a component to write data to a business entity. The user interface does not update to reflect the changed data. Use the existing Save button for writing data.
Enrich master data with data from a third-party provider	DaaS extensions	When users add or edit a business entity, the application checks the service for data related to the entity.

For more information about transformations, external resource components, and DaaS extensions, see the *Multidomain MDM Provisioning Tool Guide*.

Author

MDM Documentation Team